

Topic 31 — Alpaca Prompt Template for Instruction Tuning

Full Stack Python + AI Roadmap • Phase 4: Prompt Serialization & Instruction Formats • 3 layers: New → Intermediate → Expert



Layer 1 — New to the Concept

Alpaca is a prompt format that uses plain **text headers** to structure a prompt. It came from Stanford's Alpaca project (2023), which fine-tuned a LLaMA model to follow instructions — and the format became a widely-used standard for **instruction tuning**.

Instruction tuning = fine-tuning a base model so it's good at *following instructions* (not just completing text). You train it on thousands of **instruction** → **response** pairs. The Alpaca format is a clean layout for those pairs.

Instruction:

Summarize the following text.

Response:

Just readable text headers marking the parts. The model learns "after **### Response:**, I write the answer." Simple and human-readable — part of why it caught on.



Layer 2 — The Full Format

Alpaca has **two variants** — with and without extra input.

Variant 1 — instruction only

Below is an instruction that describes a task. Write a response that appropriately completes

Instruction:

Write a haiku about autumn.

Response:

Variant 2 — instruction + input (when there's data to act on)

Below is an instruction that describes a task, paired with an input that provides further co

Instruction:

Translate the following sentence to French.

Input:

Hello, how are you?

Response:

- **### Instruction:** — what to do (the task)
- **### Input:** — the data to act on (optional; omitted when the instruction is self-contained)
- **### Response:** — where the model writes its answer

i The preamble: the fixed intro sentence ("Below is an instruction...") is part of the official Alpaca template. There are two versions — one for instruction-only, one for instruction+input.

Building it in Python

```
def alpaca_prompt(instruction, input_text=""):
    if input_text:
        return f"""Below is an instruction that describes a task, paired with an input that

### Instruction:
{instruction}

### Input:
{input_text}

### Response:
"""
    else:
        return f"""Below is an instruction that describes a task. Write a response that app

### Instruction:
{instruction}

### Response:
"""
```



Layer 3 — Expert (why it matters & where it fits)

1. Why the format matters for fine-tuning

! Train = inference: if you fine-tune on Alpaca-formatted data, you **must** prompt in the *same* Alpaca format at inference. The model learned "instructions after **### Instruction:**, response after **### Response:** ." A different format underperforms. This is the concrete reason serialization matters — consistency between how you train and how you prompt.

2. Alpaca's place in history

Stanford Alpaca showed you could take an open base model (LLaMA) and cheaply instruction-tune it into something ChatGPT-like using ~52,000 instruction/response examples generated by GPT. The format became the *de facto* template for a wave of open-source instruction datasets and fine-tuned models — you'll see it constantly in model cards, datasets, and tutorials.

3. Single-turn, not conversational — the key limitation

Alpaca has no roles (`system` / `user` / `assistant`) and no multi-turn conversation. It's built for **single instruction** → **single response**. It doesn't naturally handle back-and-forth chat with history — which is exactly why **ChatML** (next topic) exists.

```
Alpaca    → single-turn instruction → response (no conversation history)
ChatML    → multi-turn, role-based conversations (system/user/assistant)
```

4. The dataset shape

```
{
  "instruction": "Translate to French.",
  "input": "Good morning",
  "output": "Bonjour"
}
```

During training these fields get **serialized into the Alpaca text format** (instruction/input into the template, output after `### Response:`). The JSON dataset is the structured form; the Alpaca string is its serialization. When you prepare fine-tuning data, you convert your examples into this shape.

5. Practical relevance

- Fine-tuning an open model (e.g. a specialized MSG2AI task) → you'll likely format training data in Alpaca or a similar instruction style.
- Reading open-source docs, you'll recognize `### Instruction:` / `### Response:` and know what it means.
- It clarifies *why* prompt format is a real variable, not cosmetic.

6. Modern context

Newer models often use richer chat formats (ChatML, etc.) since multi-turn chat is now the norm. But Alpaca remains foundational — simple, still widely used for single-turn instruction datasets, and understanding it makes the fancier formats easy. It's the "hello world" of instruction formats.

Interview Q&A Cards

Q: What is the Alpaca prompt format?

A: A simple instruction-tuning format using text headers — `### Instruction:` , optional `### Input:` , and `### Response:` — with a fixed preamble. From Stanford's Alpaca project; a standard for open-source instruction datasets.

Q: Why must training and inference formats match?

A: The model learns to associate structure with the exact headers seen in training. Prompting in a different format at inference removes those learned cues and degrades output — so you prompt in the same format you fine-tuned on.

Q: What's Alpaca's main limitation?

A: It's single-turn with no role structure (no system/user/assistant, no conversation history). That's why role-based formats like ChatML emerged for multi-turn chat.

Q: How does Alpaca training data look, and how is it used?

A: JSON with instruction/input/output fields. At training time those fields are serialized into the Alpaca text template (output placed after `### Response:`).

🔗 **Memory hook:** Alpaca = `### Instruction:` / `### Input:` (optional) / `### Response:` + a fixed preamble. Made for instruction tuning (single-turn instruction → response). From Stanford Alpaca. Must match train & inference format. Limitation: no roles, no multi-turn → that's why ChatML exists next.